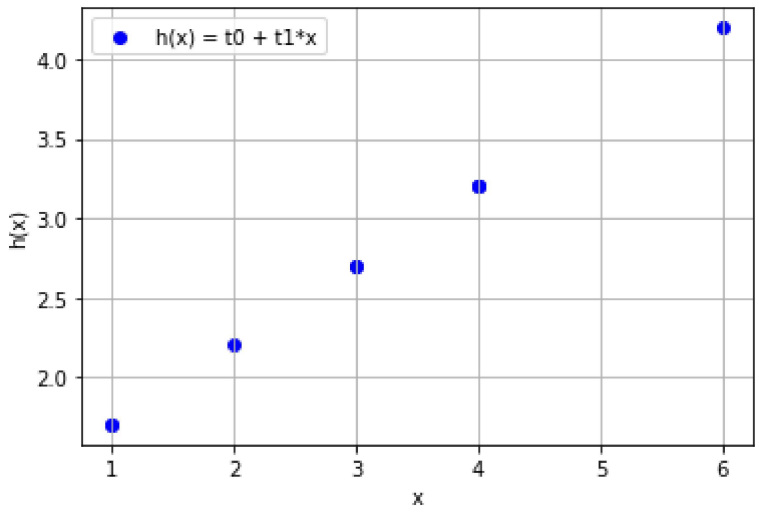
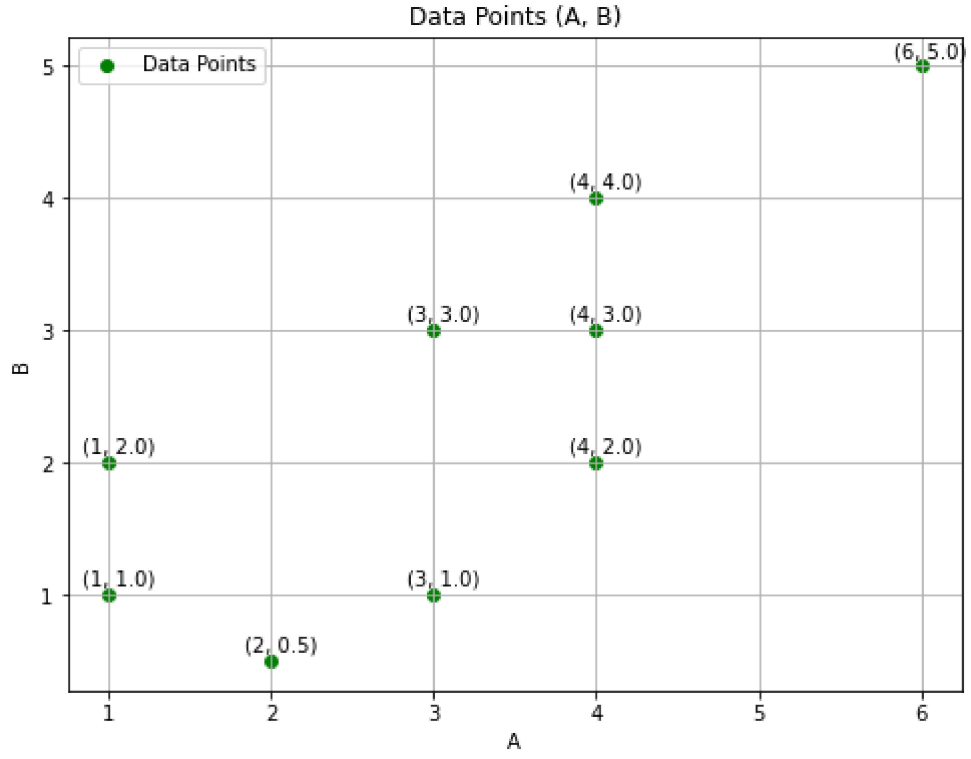
In [1]:
```python
#1
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 1, 2, 3, 4, 3, 4, 6, 4])
t0 = 1.2
t1 = 0.5
h_x = t0 + t1 * x

plt.figure(figsize=(6, 4))
plt.scatter(x, h_x, label='h(x) = t0 + t1*x', color='blue')
plt.xlabel('x')
plt.ylabel('h(x)')
plt.legend()
plt.grid(True)
plt.show()
```



In [4]:
```python
#2
import numpy as np

A = np.array([1, 1, 2, 3, 4, 3, 4, 6, 4])
B = np.array([2, 1, 0.5, 1, 3, 3, 2, 5, 4])
dot_product = np.dot(A, B)

print("Dot product:", dot_product)
```

```
Dot product: 82.0
```

In [5]:
```
#4
import numpy as np
import matplotlib.pyplot as plt

A = np.array([1, 1, 2, 3, 4, 3, 4, 6, 4])
B = np.array([2, 1, 0.5, 1, 3, 3, 2, 5, 4])

plt.figure(figsize=(8, 6))
plt.scatter(A, B, color='green', label='Data Points')
plt.xlabel('A')
plt.ylabel('B')
plt.title('Data Points (A, B)')

for i in range(len(A)):
    plt.annotate(f'({A[i]}, {B[i]})', (A[i], B[i]), textcoords="offset points", x

plt.grid(True)
plt.legend()
plt.show()
```

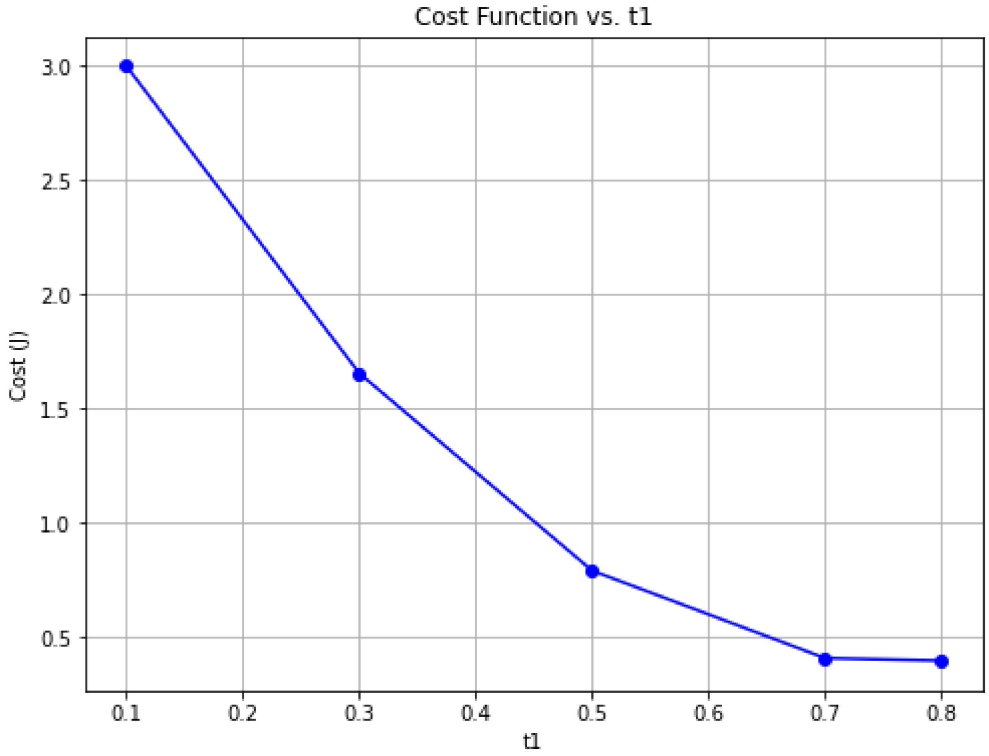


In [24]:
```
#5
import numpy as np

A = np.array([1, 1, 2, 3, 4, 3, 4, 6, 4])
B = np.array([2, 1, 0.5, 1, 3, 3, 2, 5, 4])

mse = np.mean((A - B) ** 2)
print("Mean Square Error:", mse)
```

```
Mean Square Error: 1.4722222222222223
```

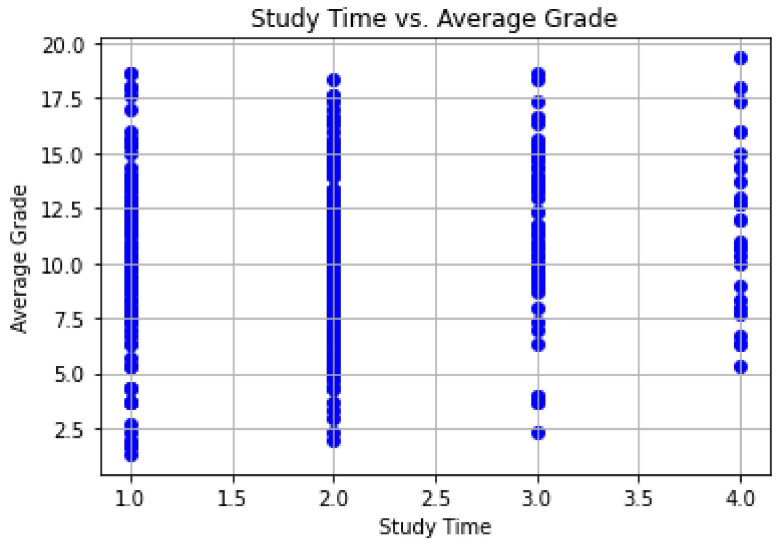
In [8]:

```
# import numpy as np
import matplotlib.pyplot as plt

# Data arrays
A = np.array([1, 1, 2, 3, 4, 3, 4, 6, 4])
B = np.array([2, 1, 0.5, 1, 3, 3, 2, 5, 4])

# Cost function definition
def compute_cost_function(t1, A, B):
    h_x = t1 * A
    sum_squared_error = np.square(h_x - B).sum()
    cost = sum_squared_error / (2 * len(A))
    return cost

# Values of t1 to test
t1_values = [0.1, 0.3, 0.5, 0.7, 0.8]
t1_list = []
cost_list = []

# Compute cost for each t1
for t1 in t1_values:
    cost = compute_cost_function(t1, A, B)
    t1_list.append(t1)
    cost_list.append(cost)

# Plotting the cost function
plt.figure(figsize=(8, 6))
plt.plot(t1_list, cost_list, marker='o', linestyle='-', color='blue')
plt.xlabel('t1')
plt.ylabel('Cost (J)')
plt.title('Cost Function vs. t1')
plt.grid(True)
plt.show()
```


Cost Function vs. t1
Cost (J)
t1
3.0
2.5
2.0
1.5
1.0
0.5
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8

In [10]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset from your local path
original_dataset = pd.read_csv("C:\\Users\\exam\\Downloads\\Students (1).csv")

# Create average grade column
original_dataset['average grade'] = (original_dataset['G1'] + original_dataset['G

# Select relevant columns
new_dataset = original_dataset[['studytime', 'average grade']]

# Save and reload the new dataset
new_dataset.to_csv('new_student_data.csv', index=False)
new_dataset = pd.read_csv('new_student_data.csv')

# Plotting
plt.scatter(new_dataset['studytime'], new_dataset['average grade'], color='blue',
plt.xlabel('Study Time')
plt.ylabel('Average Grade')
plt.title('Study Time vs. Average Grade')
plt.grid(True)
plt.show()
```



In [25]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Feature and target
X = new_dataset[['studytime']]
Y = new_dataset['average grade']

# Train-test split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_s

# Train model
regressor = LinearRegression()
regressor.fit(X_train, Y_train)

# Predict
y_pred = regressor.predict(X_test)
```

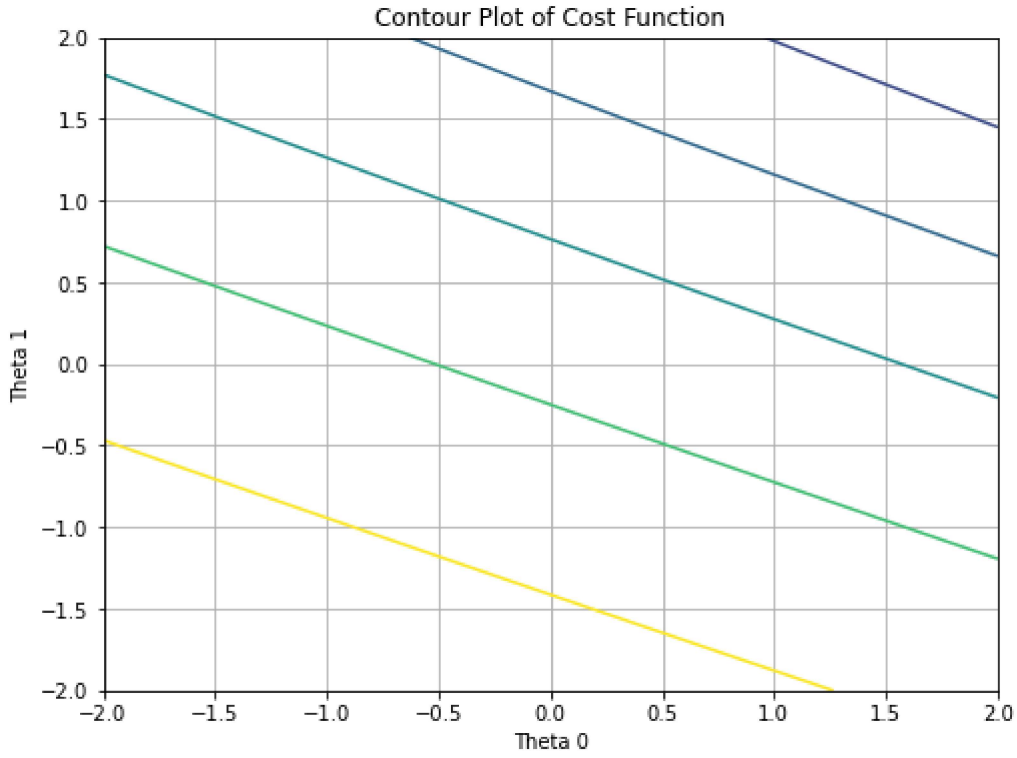
In [15]:
```
def gradient_descent(alpha, x, y, max_iter=1500):
    # Normalize feature
    x = (x - np.mean(x)) / np.std(x)
    m = len(y)
    theta = 0.0  # Initial parameter
    cost_history = []

    for _ in range(max_iter):
        h = theta * x
        error = h - y
        gradient = (1/m) * np.dot(error, x)
        theta -= alpha * gradient

        # Compute cost
        sum_squared_error = np.square(h - y).sum()
        cost = sum_squared_error / (2 * m)
        cost_history.append(cost)

    return theta, cost_history
```

In [17]:
```python
import numpy as np

def gradient_descent(alpha, x, y, max_iter=1500):
    x = (x - np.mean(x)) / np.std(x)  # Normalize feature
    m = len(y)
    theta = 0.0  # Initial parameter
    for _ in range(max_iter):
        h = theta * x
        error = h - y
        gradient = (1/m) * np.dot(error, x)
        theta -= alpha * gradient
    return theta

# Try different learning rates
learning_rates = [0.1, 0.3, 0.5, 0.7, 0.9]
learned_thetas = []

for alpha in learning_rates:
    theta = gradient_descent(alpha, new_dataset['studytime'].values, new_dataset[
    learned_thetas.append(theta)

for lr, t in zip(learning_rates, learned_thetas):
    print(f"Learning rate: {lr}, Learned theta: {t:.4f}")
```

```
Learning rate: 0.1, Learned theta: 0.4968
Learning rate: 0.3, Learned theta: 0.4968
Learning rate: 0.5, Learned theta: 0.4968
Learning rate: 0.7, Learned theta: 0.4968
Learning rate: 0.9, Learned theta: 0.4968
```

In [18]:

```
import matplotlib.pyplot as plt

# Define cost function surface
def cost_function(theta0, theta1, x, y):
    m = len(y)
    h = theta0 + theta1 * x
    return np.sum((h - y) ** 2) / (2 * m)

# Create mesh grid
theta0_vals = np.linspace(-2, 2, 50)
theta1_vals = np.linspace(-2, 2, 50)
J_vals = np.zeros((len(theta0_vals), len(theta1_vals)))

x = new_dataset['studytime'].values
y = new_dataset['average grade'].values

for i in range(len(theta0_vals)):
    for j in range(len(theta1_vals)):
        J_vals[i, j] = cost_function(theta0_vals[i], theta1_vals[j], x, y)

theta0_mesh, theta1_mesh = np.meshgrid(theta0_vals, theta1_vals)

# Plot contour
plt.figure(figsize=(8, 6))
plt.contour(theta0_mesh, theta1_mesh, J_vals.T, levels=np.logspace(-1, 2, 20))
plt.xlabel('Theta 0')
plt.ylabel('Theta 1')
plt.title('Contour Plot of Cost Function')
plt.grid(True)
plt.show()
```



In [19]:

```python
from sklearn.model_selection import KFold, RepeatedKFold
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.linear_model import LinearRegression

X = new_dataset[['studytime']]
y = new_dataset['average grade']

kf = KFold(n_splits=5, shuffle=True, random_state=42)
rkf = RepeatedKFold(n_splits=5, n_repeats=3, random_state=42)

def evaluate_model(cv):
    mae_list, mse_list, rmse_list, me_list = [], [], [], []
    for train_index, test_index in cv.split(X):
        X_train, X_test = X.iloc[train_index], X.iloc[test_index]
        y_train, y_test = y.iloc[train_index], y.iloc[test_index]
        model = LinearRegression()
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        mae_list.append(mean_absolute_error(y_test, y_pred))
        mse_list.append(mean_squared_error(y_test, y_pred))
        rmse_list.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        me_list.append(np.mean(y_test - y_pred))
    return {
        'MAE': np.mean(mae_list),
        'MSE': np.mean(mse_list),
        'RMSE': np.mean(rmse_list),
        'ME': np.mean(me_list)
    }

print("Simple K-Fold:", evaluate_model(kf))
print("Repeated K-Fold:", evaluate_model(rkf))
```

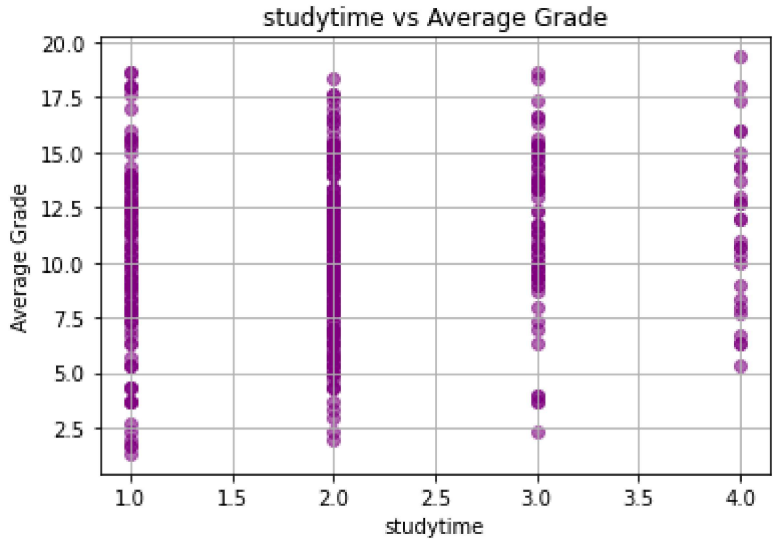
```
Simple K-Fold: {'MAE': 2.9745546919447974, 'MSE': 13.56412173956085, 'RMSE': 3.
675617187079309, 'ME': -0.006845541486901136}
Repeated K-Fold: {'MAE': 2.9737284273252556, 'MSE': 13.56392427604257, 'RMSE':
3.6754325913276715, 'ME': -0.002608468058498866}
```

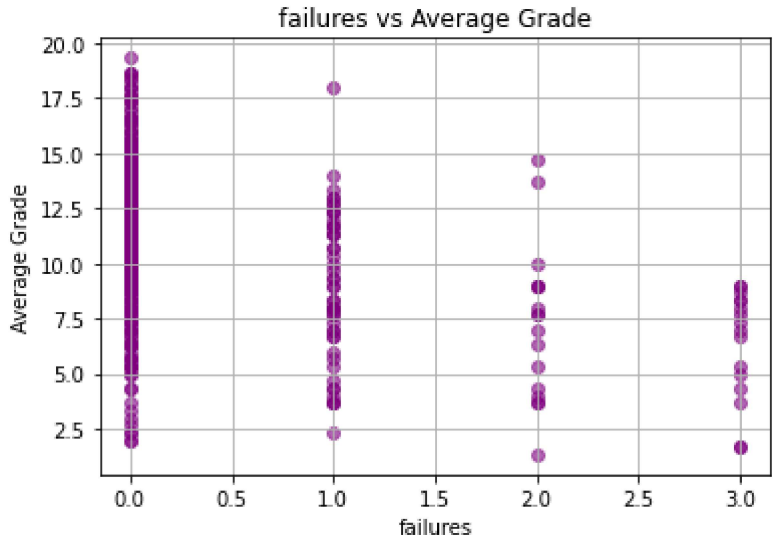
In [22]:

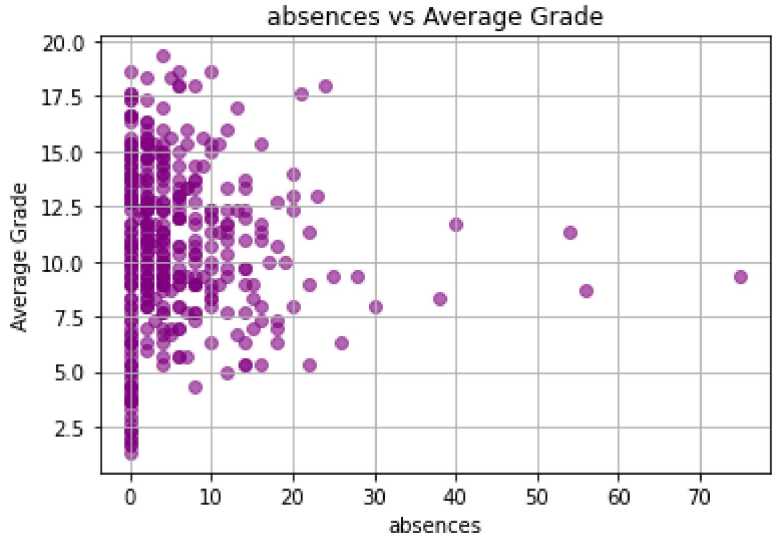
```python
#PART C
#1. Implement gradient descend for multivariate linear regression to fit data in
import numpy as np
import pandas as pd


data = pd.read_csv("C:\\Users\\exam\\Downloads\\Students (1).csv")


data['average grade'] = (data['G1'] + data['G2'] + data['G3']) / 3


features = data[['studytime', 'failures', 'absences']]  # Example features
target = data['average grade']


X = (features - features.mean()) / features.std()
X = np.c_[np.ones(X.shape[0]), X]  # Add intercept term
y = target.values
m, n = X.shape

# Gradient Descent Function
def gradient_descent(X, y, alpha=0.01, max_iter=1500):
    theta = np.zeros(n)
    cost_history = []

    for _ in range(max_iter):
        h = np.dot(X, theta)
        error = h - y
        gradient = (1/m) * np.dot(X.T, error)
        theta -= alpha * gradient

        cost = np.sum(error ** 2) / (2 * m)
        cost_history.append(cost)

    return theta, cost_history

# Run gradient descent
theta, cost_history = gradient_descent(X, y, alpha=0.01)

print("Learned parameters (theta):", theta)
```

```
Learned parameters (theta): [10.67932187  0.26863136 -1.34762775  0.0808781 ]
```

In [23]:
```
#2. Analyze impact of each input variable on the output variable average grade(gi
import matplotlib.pyplot as plt

input_features = ['studytime', 'failures', 'absences']

for feature in input_features:
    plt.figure(figsize=(6, 4))
    plt.scatter(data[feature], data['average grade'], color='purple', alpha=0.6)
    plt.xlabel(feature)
    plt.ylabel('Average Grade')
    plt.title(f'{feature} vs Average Grade')
    plt.grid(True)
    plt.show()
```






absences vs Average Grade
Average Grade
absences
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
10
20
30
40
50
60
70

In [ ]: